

INFORME TÉCNICO — ACTIVIDAD DE EVALUACIÓN 2 (AE2)

Proyecto: "Mercado Fresco" (Almacén de Barrio)

Asignatura: Paradigmas y Lenguajes de Programación III

Estudiante: Tomás Romero Baylac

Enlace al GitHub:

<https://github.com/tomasromerobaylac/AE2-Paradigmas-y-Lenguajes-de-Programacion-3>

Introducción

Este informe detalla la implementación técnica y el fundamento metodológico correspondientes a la Actividad de Evaluación 2 (AE2) de la asignatura Paradigmas y Lenguajes de Programación III. El desarrollo se centra en la evolución de la aplicación web del proyecto "Mercado Fresco" desde un maquetado estático (TP1) hacia una arquitectura interactiva basada en JavaScript Vanilla, incorporando un panel interno de inventario que aplica programación asíncrona, gestión dinámica del DOM y una metodología de diseño web de la Unidad 1.

1. Alcance Técnico y Requerimientos de JavaScript (Opción 12)

Para esta entrega se seleccionó la **Opción 12: Monitor de Inventario y Alerta de Estado Crítico**, incorporando al proyecto un panel interno (`inventario.html`) destinado al personal del almacén, que muestra el stock en tiempo real y alerta visualmente cuando un producto cae por debajo de su umbral crítico. Los requerimientos técnicos transversales implementados abarcan:

1. **Desacoplamiento de Eventos (`addEventListener`):** se eliminaron todos los controladores intrusivos (`onclick`, `onsubmit`) del marcado HTML. Toda la interacción — refrescar el panel, cambiar de rol, filtrar por categoría o buscador, y ajustar cantidades con los botones `+/-` generados dinámicamente — se maneja desde `app.js` mediante `addEventListener()`, incluyendo delegación de eventos para los controles que se crean después de la carga inicial.
2. **Asincronía y Llamadas HTTP (`fetch` + `async/await`):** se diseñó un recurso local en formato JSON (`stock.json`) que simula la respuesta de un servidor con el stock de 16 productos y su umbral crítico. Una función asíncrona (`cargarInventario`) consulta ese recurso con `fetch()` y `await`, sin bloquear el hilo principal, y maneja errores de red con `try/catch`.
3. **Mutación Dinámica del DOM:** la tabla de inventario se genera íntegramente por JavaScript a partir de los datos recibidos. Cada fila cuya cantidad es inferior a su umbral se marca con la clase `stock-critico` y un badge de alerta; un resumen y un banner superior se recalculan en cada carga o ajuste de stock.

2. Fundamentación Metodológica (Unidad 1 — VSDM)

El diseño e implementación de este módulo responde a los lineamientos de la metodología **VSDM (Vistas Separadas por rol de acceso)**:

- **Justificación conceptual:** VSDM plantea que una misma pantalla puede exponer distintas vistas de la información según el rol de quien la consulta, sin duplicar la estructura de datos subyacente. En "Mercado Fresco", el panel de inventario es utilizado tanto por un cliente que solo necesita saber si un producto está disponible, como por un empleado que necesita cantidades exactas y control operativo.
- **Modelado de las vistas:** un selector de rol (Cliente / Empleado) alterna una clase (rol-empleado) sobre <body>. La vista Cliente solo informa disponibilidad, ocultando cantidades y controles (columna .col-empleado); la vista Empleado expone la cantidad exacta, el umbral configurado y los botones +/- para ajustar stock.
- **Separación de responsabilidades:** la arquitectura distingue la capa de datos (stock.json, consumido vía fetch), la lógica de comportamiento (escuchadores desacoplados y la función aplicarRol()) y la representación visual (CSS condicionado por la clase de rol sobre <body>).
- **Alcance de la simulación:** el cambio de vista ocurre enteramente del lado del cliente, sin autenticación real — se documenta explícitamente dentro de inventario.html que en una versión con backend este control debería resolverse con autenticación y autorización en el servidor, no solo en el navegador.

3. Estructura de Archivos y Código Implementado

A. Archivos del módulo (AE2)

Archivo	Rol
inventario.html	Panel interno: estructura, selector de rol, tabla de stock
app.js	Lógica: fetch()+async/await, addEventListener(), render del DOM, VSDM
stock.json	Recurso local que simula la respuesta del servidor (16 productos)

El módulo reutiliza además styles.css, products.js y cart.js del resto del sitio (header, navegación y badge del carrito compartidos), ya que inventario.html es una pantalla más del mismo proyecto de Mercado Fresco y no una aplicación aislada.

B. Extracto del código clave — Asincronía y eventos (app.js)

```

async function cargarInventario() {
  mostrarEstado("Consultando stock...");
  try {
    const respuesta = await fetch(STOCK_URL);
    if (!respuesta.ok) {
      throw new Error("El servidor respondió con estado " + respuesta.status);
    }
    const datos = await respuesta.json();
    inventario = datos;
    renderInventario();
    mostrarEstado("Inventario actualizado · " + new Date().toLocaleTimeString("es-AR"));
  } catch (error) {
    mostrarEstado("No se pudo cargar el inventario. Revisá la conexión e intentá de nuevo.", true);
    console.error("Error al cargar stock.json:", error);
  }
}

function esCritico(item) {
  return item.cantidad < item.umbral;
}

```

```
}
```

C. Extracto del código clave — VSDM y delegación de eventos (app.js)

```
function aplicarRol(rol) {
  rolActual = rol;
  document.body.classList.toggle("rol-empleado", rol === "empleado");
  document.getElementById("rol-explicacion").textContent =
    rol === "empleado"
      ? "Vista Empleado: se muestran las cantidades exactas y los controles para ajustar stock."
      : "Vista Cliente: solo se informa si el producto está disponible, sin cantidades ni controles inter
}

function inicializar() {
  document.getElementById("btn-refrescar").addEventListener("click", cargarInventario);
  document.getElementById("selector-rol").addEventListener("change", (e) => aplicarRol(e.target.value));
  document.getElementById("filtro-categoria").addEventListener("change", renderInventario);
  document.getElementById("buscador").addEventListener("input", renderInventario);

  // Delegacion de eventos: los botones +/- se generan dinamicamente al renderizar
  document.getElementById("cuerpo-inventario").addEventListener("click", (evento) => {
    const boton = evento.target.closest("[data-stock-action]");
    if (!boton) return;
    const delta = boton.dataset.stockAction === "sumar" ? 1 : -1;
    actualizarCantidad(boton.dataset.id, delta);
  });

  aplicarRol(document.getElementById("selector-rol").value);
  cargarInventario();
}

document.addEventListener("DOMContentLoaded", inicializar);
```

4. Verificación y pruebas

El módulo fue probado con una batería de pruebas automatizadas sobre el DOM real (jsdom, servido por un servidor HTTP local) que verificó: la carga asíncrona del stock, el conteo correcto de los 5 productos en estado crítico, la coherencia entre el valor inicial del selector de rol y la clase aplicada al body (VSDM), el comportamiento de los botones +/- y la ausencia de atributos onclick/onsubmit en el HTML. Las pruebas corrigieron un desajuste real detectado entre el rol inicial seleccionado en el HTML y el estado inicial en JavaScript.

Conclusión

La realización de esta actividad permitió consolidar los conocimientos teóricos y prácticos en el desarrollo de aplicaciones web interactivas. La combinación de JavaScript moderno con eventos desacoplados, llamadas asíncronas limpias con fetch y el rigor conceptual de la metodología VSDM permite que un mismo panel de inventario sirva, de forma coherente, tanto a la vista simplificada del cliente como a la vista operativa del personal del almacén.